

01

单元测试



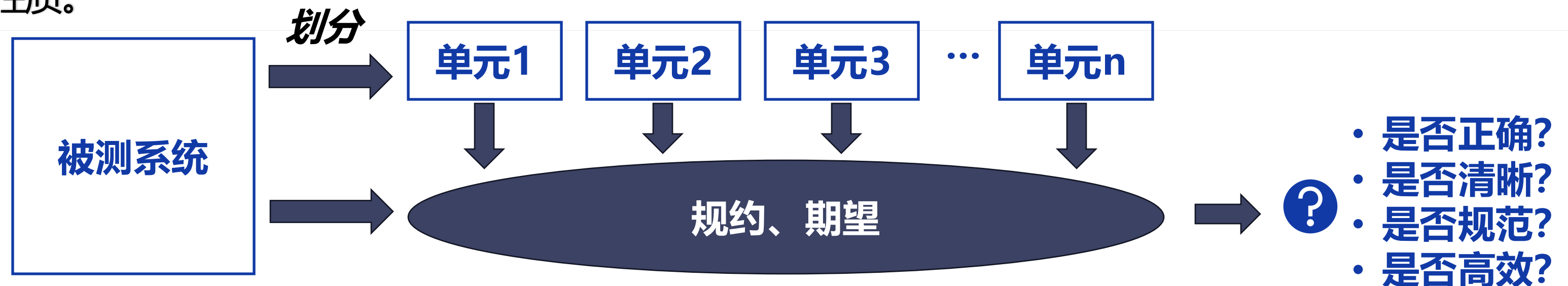
单元测试概述

单元测试的定义

单元测试构成了所有其他测试构建的基础。是一种针对应用程序代码分解为组件的测试技术，验证每个块或单元的相关数据、使用过程和功能，以确保每个块按预期工作。单元测试的准确性和彻底性是影响其他测试的执行情况以及软件整体性能的重要因素。

单元测试的目的

其目的是检测判断每个程序模块的行为是否与期望一致。合格的代码应具备正确性、清晰性、规范性、高效性等性质。



开发者测试 ■ 单元测试

表6.1所示是一个简单的计算器类，此例的类 Calculator有个 Add 方法看起来非常可靠，但仍然需要测试它。为此，添加一个新的项目并为其提供对包含计算器的项目的使用。在计算器测试器的主体中执行以下操作，这就是一个极其简单的单元测试，如表6.2所示。为了更好的维护性，创建一个名为 “Adding12And11” 的方法，如表6.3所示。

表 6.1: 计算器程序待测代码

```
1 public class Calculator{
2     public int Add(int x, int y){
3         return x + y;
4     }
5 }
```

表 6.2: 计算器程序测试代码

```
1 class Program{
2     static void Main(string[] args){
3         var calculator = new Calculator();
4         int result = calculator.Add(12, 11);
5         if (result != 23)
6             throw new InvalidOperationException();
7     }
8 }
```

表 6.3: 计算器程序单元测试代码

```
public class CalculatorTests{
    [TestMethod]
    public void Adding12And11(){
        var calculator = new Calculator();
        int result = calculator.Add(12, 11);
        Assert.AreEqual(23, result);
    }
}
```

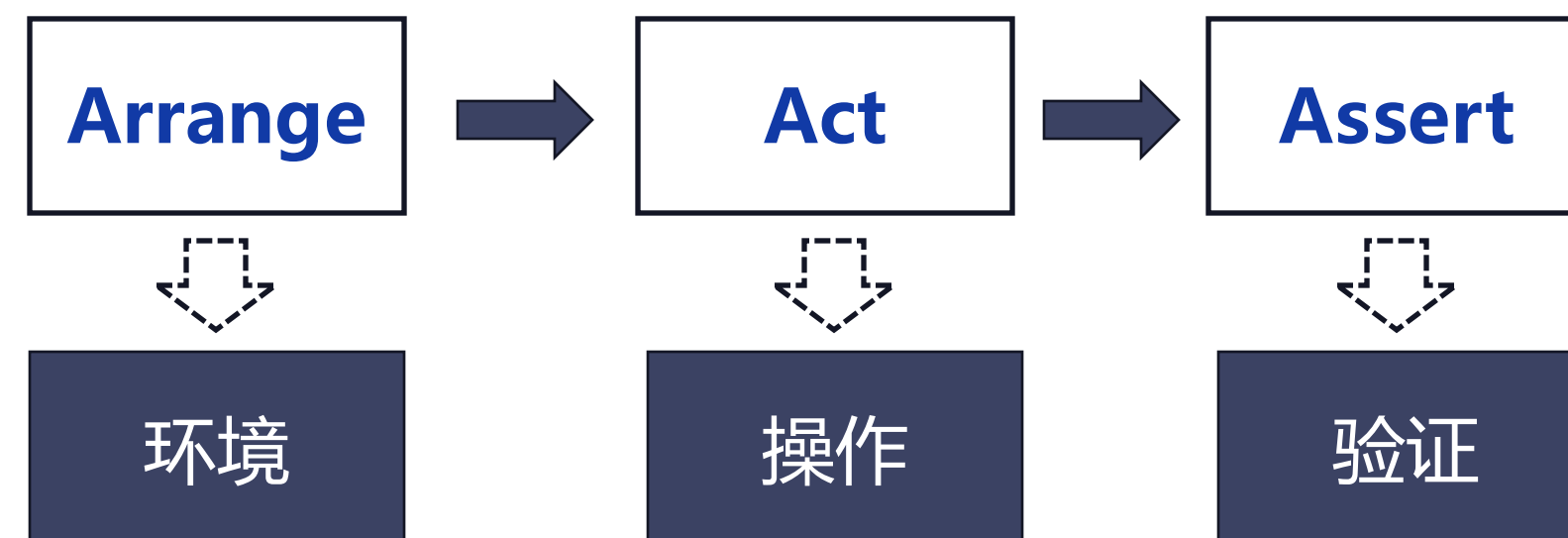
单元测试 3A 原则

什么是单元测试3A原则

单元测试的 3A 原则是一种指导单元测试编写的规范，它将测试过程分为三个清晰的步骤：
Arrange (准备)、Act (执行)、Assert (断言)。

单元测试3A原则的作用

3A 原则将突出显示调用代码所需的依赖项、代码的调用方式以及尝试断言的内容。



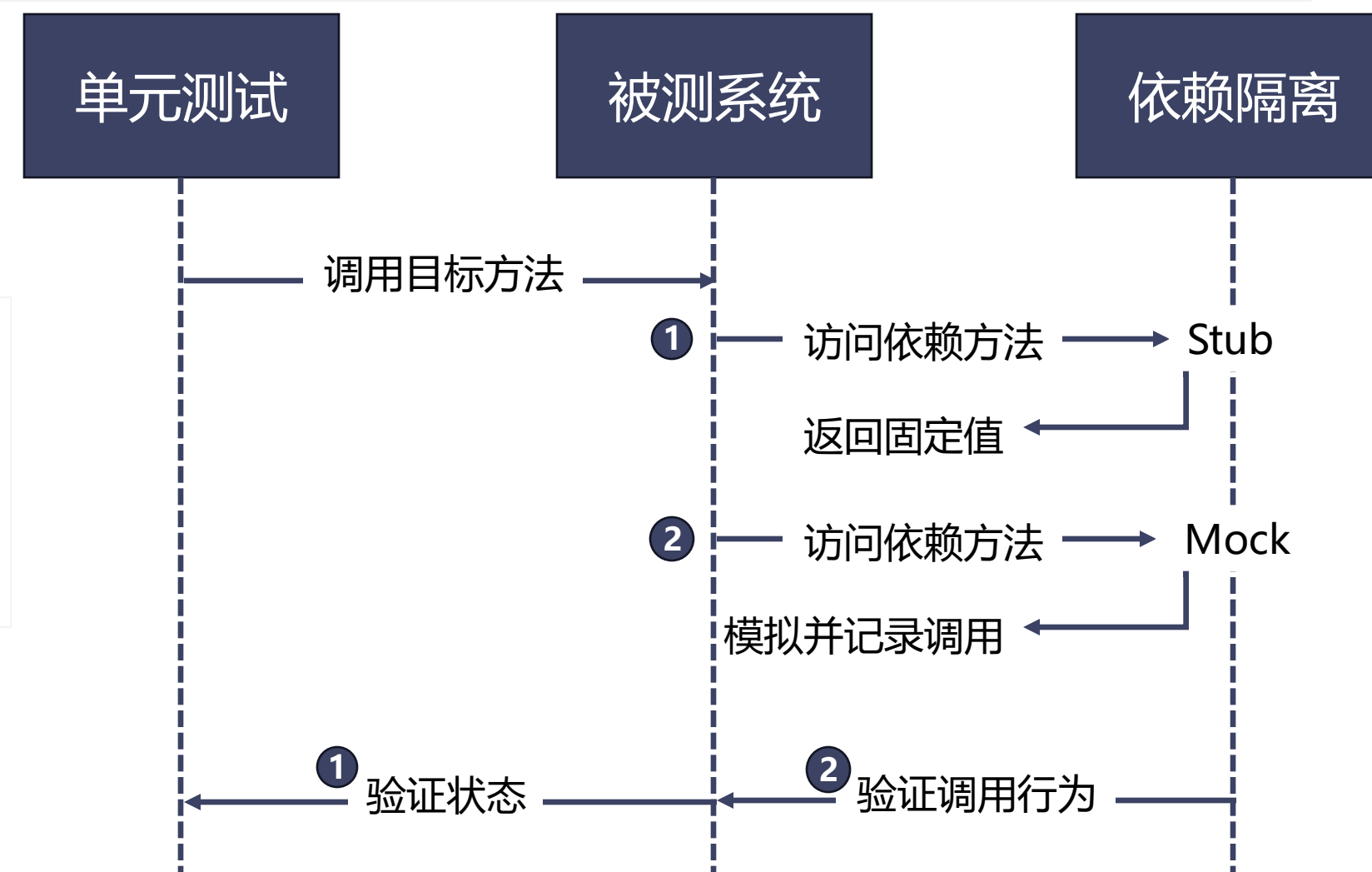
依赖隔离技术概述

关于依赖隔离技术

单元测试常常需要隔离源文件中的函数或过程，以便独立测试这些小代码单元。为了隔离代码单元，开发者需要额外的支撑技术。最常用的依赖隔离技术是模拟（Mock）和存根（Stub）。

依赖隔离的两种技术

- 模拟 (Mock): 使用虚假业务逻辑的模拟对象代替真实对象，隔离外部依赖，方便测试代码的独立验证。
- 存根 (Stub): 使用预先确定的行为代替真实行为，实现抽象类或接口，通常用于客户端期望相同响应的场景。



尽早化单元测试

进行单元测试的时机

单元测试是开发周期中首先进行的，通常由开发者完成。尽早开展单元测试并实现自动化是关键。测试驱动开发 TDD 可以在编写实际代码之前创建测试，然后编写代码以通过这些测试。

尽早化单元测试的优势

- 尽早发现问题：降低修复成本，提高代码质量。
- 确保代码可测试：TDD 可以帮助开发者设计可测试的代码结构。
- 提高代码质量：通过测试验证代码的正确性和健壮性。

自动化单元测试



什么是自动化单元测试

自动化单元测试是指使用自动化测试工具或框架来执行单元测试的过程。它将测试代码的执行和结果验证自动化，无需人工干预。

自动化单元测试的优势

- 提高测试效率
- 支持持续集成和持续部署 (CI/CD)
- 辅助新手编写高质量的测试
- 减少人为错误
- 集中精力测试复杂代码
- 生成多指标代码覆盖率分析

简单化单元测试

什么是简单化单元测试？

单元测试代码也可能存在错误，尤其是在复杂性较高的情况下。应该为每个测试方法设置独立的断言，以方便后期单元测试的分析与故障诊断。

如何使单元测试简单化？

- 测试代码应尽可能简单：以减少错误和提高可维护性。
- 每个测试方法只测试一个功能或行为：并使用独立的断言，便于故障诊断。
- 避免在一个测试方法中测试太多内容：以防测试代码变得复杂和难以维护。
- 测试的数量并非总是关键：更重要的是每个测试方法应专注于一个特定的功能或行为。

高质量单元测试

高质量单元测试示例

当待测应用程序设计用于处理金融交易，则测试数据应包括实际交易金额、日期和客户信息。开发者还应该测试边界条件，例如输入变量的最小值和最大值。通过测试正面和负面场景，可以确保代码稳健并且可以处理意外情况。

如何设计高质量单元测试方法

- 使用实际数据： 模拟真实场景，如金融交易的金额、日期和客户信息。
- 测试边界条件： 包括输入变量的最小值和最大值。
- 覆盖正面和负面场景： 测试代码在正常情况和异常情况下的行为。
- 采用多样性测试和基于故障假设测试： 在缺乏领域经验时，生成高质量的测试数据。
- 结合辅助工具进行自动化测试数据生成： 提高测试效率和数据质量。

单元测试的其他优势

- 提供不同视角：帮助开发者从用户或其他组件的角度审视代码，发现潜在问题。
- 作为额外代码审查：确保代码在首次编写时就是正确和健壮的。
- 考虑代码接口的使用场景：提前发现并修复可能在软件测试后期阶段暴露的问题。
- 降低缺陷修复成本：在软件生命周期的早期阶段发现缺陷，修复成本远低于在后期发现缺陷的成本。

思考：

- 除了上述优势还有没有其他优势？

